



Agent Architecture Decisions

Choosing the Right Pattern

BY PROF. PETERSON

Course Overview

01

Why Architecture Matters

03

Multi-Purpose Agent

05

Orchestrated Design

02

Single-Purpose Agent

04

Multi-Agent System

06

Tradeoffs, Framework & Lab



About Prof. Peterson

1



Welder

2



Systems Engineer

3



Professor & Researcher

Teaching: **AI • Data Mining • MIS • Cybersecurity**

Why Architecture Choices Matter

Foundation of Everything

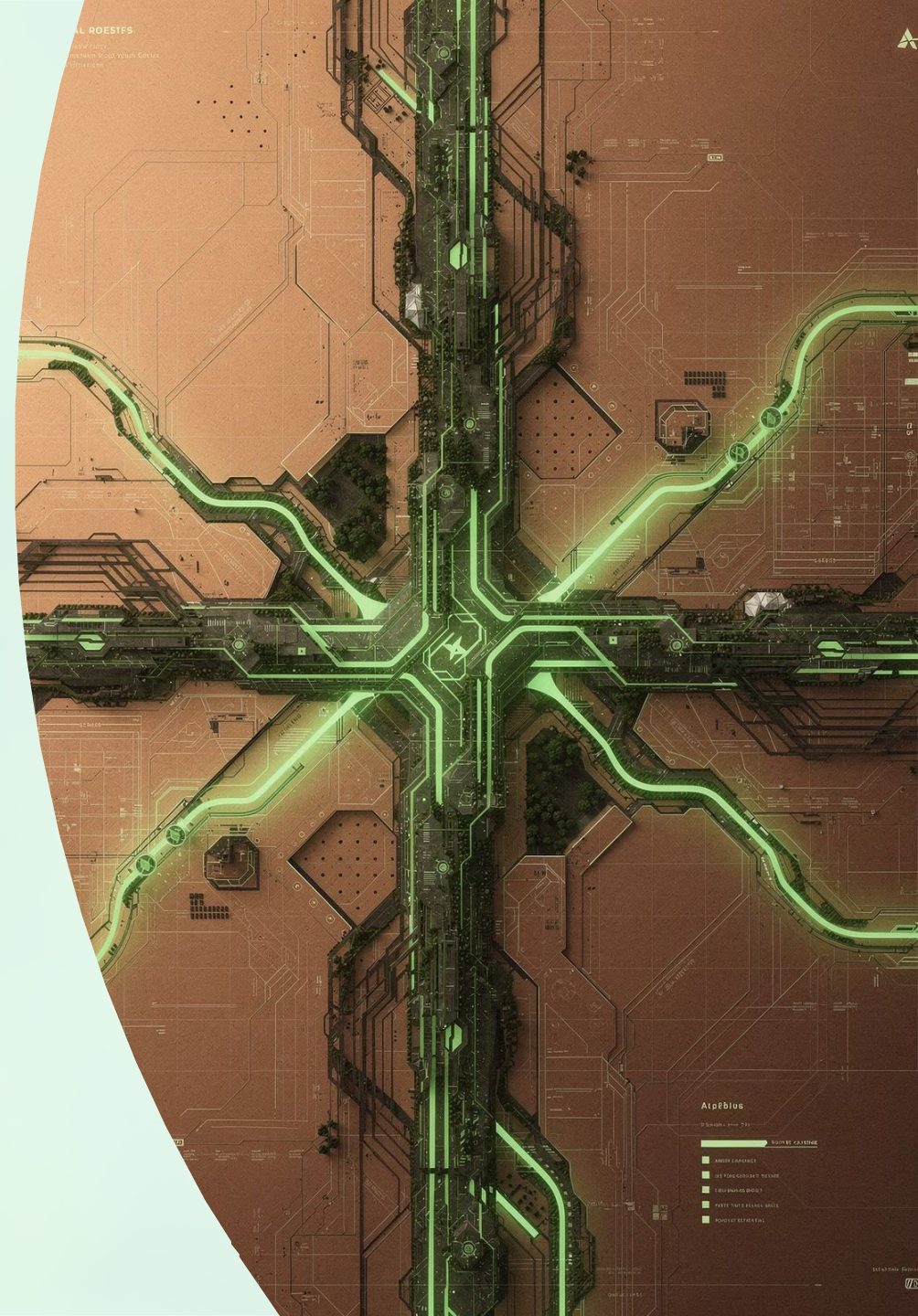
The architecture you choose shapes cost, speed, reliability, and safety from day one.

Hard to Change Later

Poor early decisions compound — refactoring agent architecture mid-project is expensive and risky.

No One-Size-Fits-All

Each pattern has strengths. Matching the right pattern to the problem is the core skill.



Single-Purpose Agent

ONE CLEAR JOB 🎯



What It Is

An agent designed to do **exactly one thing** — and do it well. Narrow scope, clear inputs and outputs, minimal ambiguity.

Best For

- Well-defined, repeatable tasks
- High-reliability requirements
- Low-latency, low-cost deployments

Multi-Purpose Agent

THE "DO-EVERYTHING" BOT 🤖

What It Is

A single agent handling **multiple task types** through broad instructions or tool access. Flexible but complex.

Watch Out For

- Prompt bloat and confusion
- Harder to debug and test
- Reliability drops as scope grows





Multi-Agent System

TEAM OF SPECIALISTS 

Specialist Agents

Each agent owns a narrow domain — research, writing, validation, etc.

Parallel Execution

Agents can work simultaneously, reducing total task time.

Modular Design

Swap or upgrade individual agents without rebuilding the whole system.

Orchestrated Design

ADDING A CONDUCTOR 



What It Adds

An **orchestrator agent** coordinates the team — routing tasks, managing state, and synthesizing outputs.

Key Benefits

- Centralized control and visibility
- Dynamic task routing
- Handles complex, multi-step workflows



Discussion: Your AI Systems

Where and what are the AI systems in your world?

At Work

What AI tools or agents does your organization already use?

In Products

Are any of your systems AI-powered under the hood?

In Your Life

What personal AI assistants or agents do you rely on daily?

Tradeoffs to Consider



Latency 🕒

More agents = more hops. Orchestration adds coordination overhead.



Reliability 🛡️

More components = more failure points. Simpler systems fail more gracefully.



Cost 💰

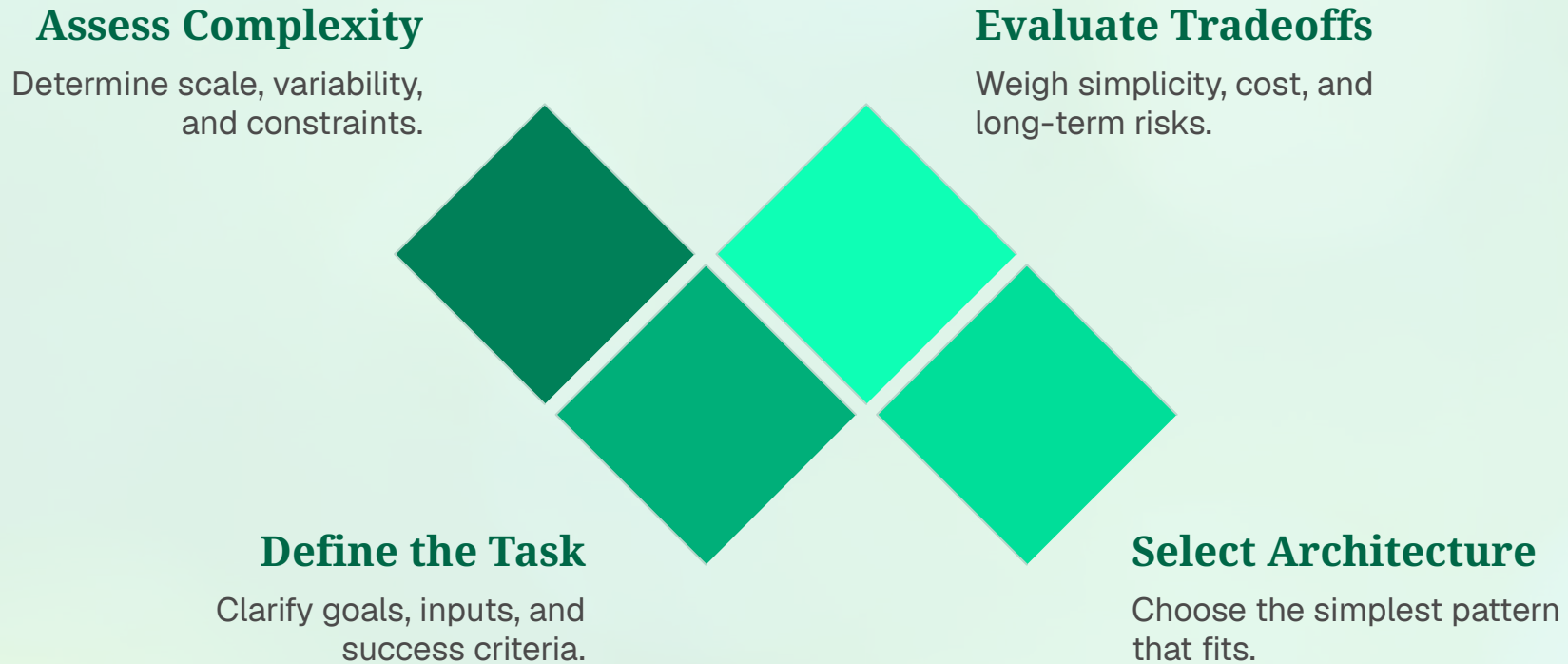
Each LLM call costs tokens. Multi-agent systems multiply API usage quickly.



Safety ⚠️

Autonomous agents can take unintended actions. Guard rails are essential.

How to Choose the Right Pattern



Start with the simplest architecture that solves the problem. Only add complexity when the task genuinely demands it — not because it's impressive.



When Single-Agent Wins



Task is Well-Defined

Clear inputs, predictable outputs, minimal branching logic.

Speed Matters

No coordination overhead — lowest latency path to an answer.

Budget is Tight

One agent, one call — the most cost-efficient option available.

When Multi-Agent Wins 🌟

Complex, Parallel Tasks

Work can be split and run simultaneously across specialists.

Domain Expertise Needed

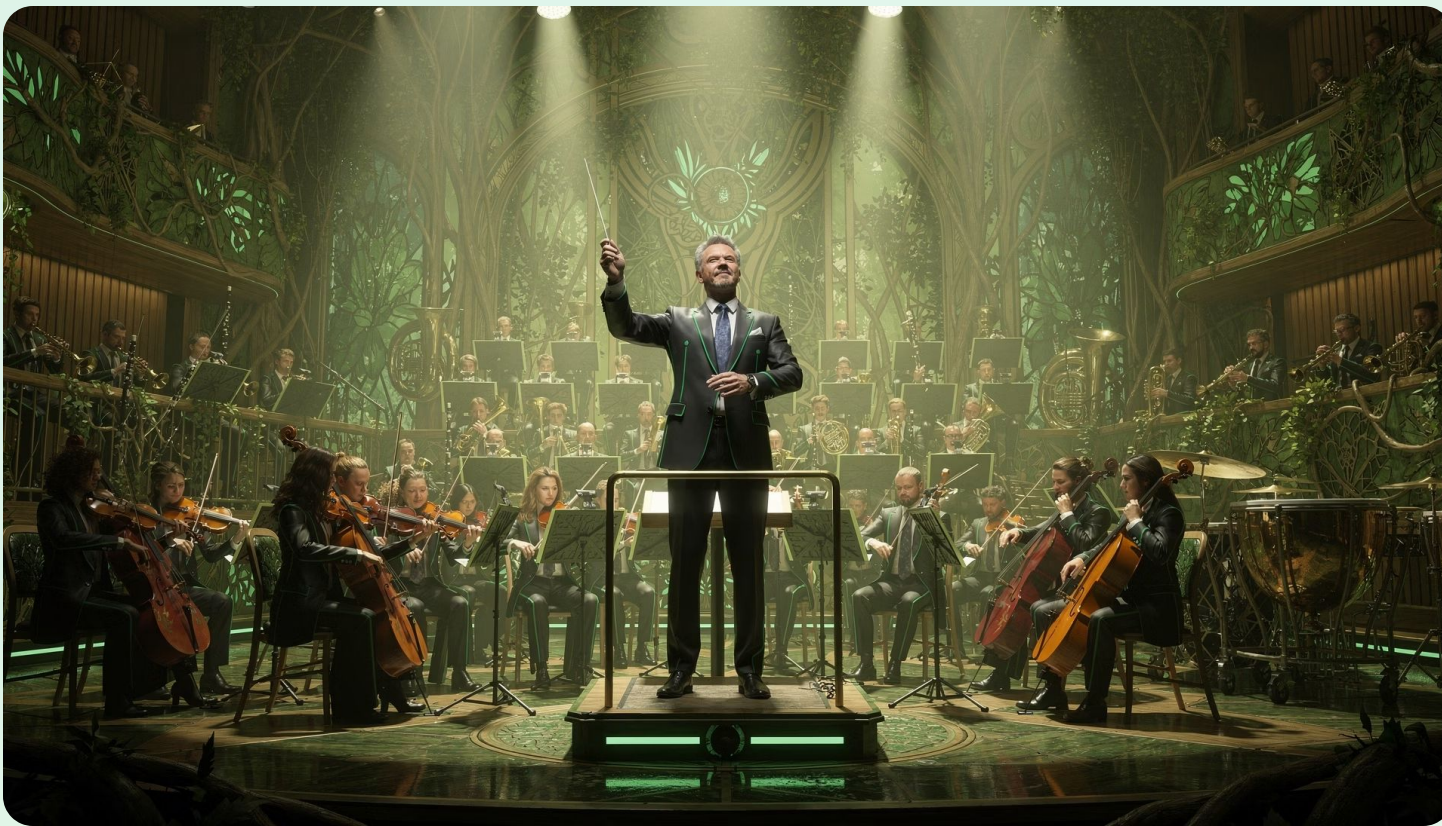
Different subtasks require different tools, prompts, or models.

Scale & Modularity

System needs to grow — add agents without rebuilding everything.



When Orchestration Is Worth It 🎻



Use Orchestration When:

- Tasks are **dynamic** — routing decisions depend on prior outputs
- State must be **maintained** across multiple agent interactions
- You need **centralized logging**, monitoring, or human-in-the-loop checkpoints
- Failure recovery requires **intelligent retry logic**

Architecture Cracks & Red Flags



Prompt Overload

One agent doing too many jobs — instructions become contradictory.



Silent Failures

Agents fail without surfacing errors — outputs look fine but are wrong.



Runaway Loops

Agents calling each other indefinitely with no exit condition.



No Guard Rails

Autonomous agents taking real-world actions without human oversight.



Hands-On Scenario

PICK YOUR ARCHITECTURE 



The Challenge

You're building an AI system for a real use case. Given the constraints below — which architecture do you choose?

- Budget: Limited API calls per day
- Task: Multi-step research + summarization + email draft
- Users: Non-technical stakeholders
- Reliability: Must not hallucinate facts



Group Discussion & Redesign

💬 Share your architecture choice — then redesign it with a different pattern. What breaks? What improves?

Present Your Choice

Explain which pattern you picked and why.

Stress Test It

What happens when volume doubles? When an agent fails?

Redesign

Swap the architecture — what tradeoffs emerge?

Practical Decision Checklist

1 Is the task well-defined with clear inputs/outputs?

→ If yes, start with a single-purpose agent.

2 Does it require parallel or specialized subtasks?

→ If yes, consider a multi-agent system.

3 Is dynamic routing or state management needed?

→ If yes, add an orchestrator.

4 Have you accounted for latency, cost, and safety?

→ Always evaluate all four tradeoffs before finalizing.



Closing Takeaways 🎉

Start Simple

Default to the least complex architecture that solves the problem.

Match Pattern to Problem

No architecture is universally best — context is everything.

Design for Failure

Build in guard rails, logging, and human checkpoints from the start.

Iterate Boldly

The best architecture evolves — revisit decisions as your system grows.

Connect with Prof. Peterson



GitHub

Thatguywiththeshirt



LinkedIn

/in/petersonchase/



Email

chase.peterson@metrostate.edu

chase@chase.nexus